



Технологии Программирования

Скрытая Угроза
(Написание надежного кода)

Ордена



Quality

- Надежный код
- Качественное ревью (ревью митинг)
- 100% покрытие тестами CodeCov100
- Требование к прохождению тестов PassRate100
- Многоуровневое тестирование/FuSa
- Регулярная валидация
- Код без багов Zero Bug tolerance
- Многокритериальный релиз

TTM

- Упор на фичи
- Продукт модифицируется под требования пользователя
- Тесты - не продукт
- Ручная валидация Manual test specs
- Прототип в релиз
- Релиз вне цикла
- Короткий релизный цикл



Есть только ROI

Вопрос!

РЕГУЛЯРНАЯ РУЧНАЯ ВАЛИДАЦИЯ

Цена
Ошибки

System
test

Тесты
Интеграции

Юнит-тесты

Надежный код

Стоимость
стадии

Велика мудрость в коде том

Понятные и лаконичные имена функций и переменных

- CalculateScreenSize - функция, что делает
- m_screenWidth - переменная, что хранит

Атомарные функции

- Функция выполняет только одну конкретную задачу
- Избегать функций “всемогуторов”
- Длина функции не более одного экрана
- Избегать множественной вложенности, 3 уровня максимум
- Любые внешние данные должны быть проверены

Исполнение максимально линейное

- Избегать запутанной логики исполнения
- Одна строка - одна операция
- Минимизировать goto
- Избегать вложенных лямбд с отложенным вызовом

Понятные комментарии

- Описывается должен метод решения

Код должен быть написан так, чтобы он был понятен даже через месяц



Компилятор - великий союзник твой

То чего не должно случиться, случиться не должно!

- const reference - если не возможен nullptr
- unsigned type - если отрицательный тип нежелателен
- Размерные типы: uint64_t, int32_t
- Контракты

Архитектура системы обработки ошибок

- Error codes
- Exceptions

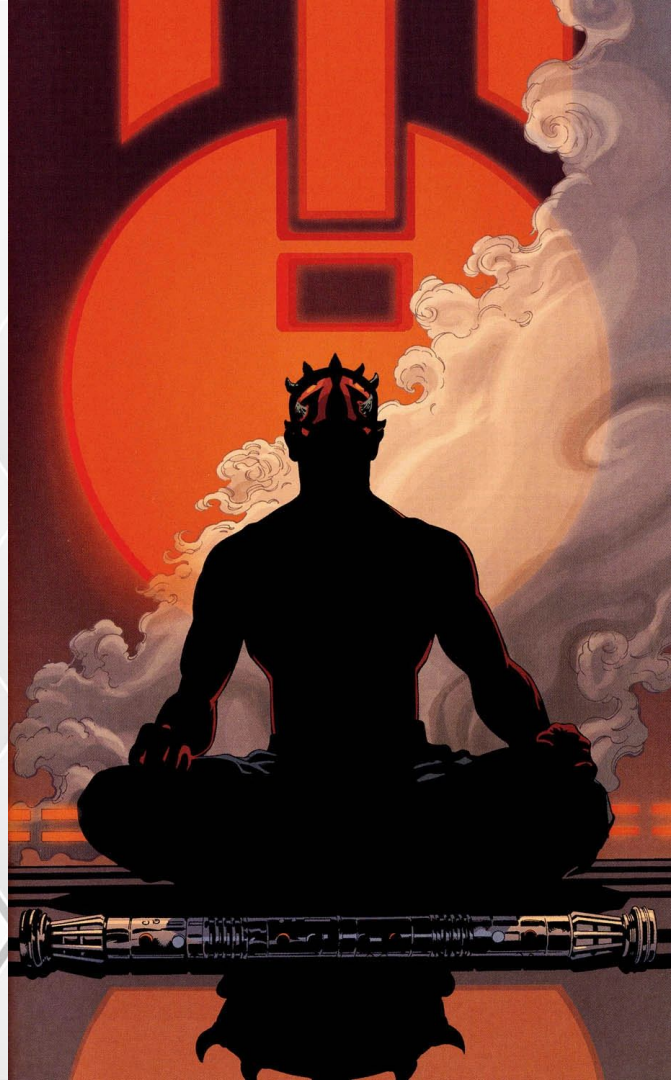
Trust no one!

- Любые внешние данные должны быть проверены

Максимальное использование компилятора

- -W4 - любая мелочь должна быть починена
- -Wall - максимально придирчивая компиляция
- -Werror - любая мелочь - ошибка
- Не использовать #pragma warning

Если существуют два способа сделать что-либо, причём один из них ведёт к катастрофе, то кто-нибудь изберет именно этот способ (“Эдвард Мерфи”)



Вопрос*

sizeof(int) = **2B** (sega, Arduino uno 1), **4B** (IA86, AMD64), **8B** (Itanium)

sizeof(size_t) = **sizeof(void*)** = **4B** (IA32), **8B** (AMD64, Itanium)

Выравнивание структур, сетевые протоколы, CPU<->GPU...

Взору моему открылось будущее кода

Задачи ревью кода:

- Реализация правила четырех рук
- Новый взгляд на код и задачу
- Проверка корректности алгоритма
- Поиск алгоритмических недочетов
- Поиск мелких кодовых недочетов и очевидных багов, исключение проблемы “замыленности”

Не надо писать “ваш” код чужими руками

Код ревью можно облегчить использованием статических анализаторов кода. Klockwork, PVS Studio, Coverity...

Ревью - это не проверка на запятые и именование!

Для этого есть автоматизированные инструменты.



Двое их всегда, учитель и ученик. XP

Менторство/Обучение

- Подходит для обучения нового сотрудника
- Наличие старшего программиста в паре необходимо

Взаимное ревью

- Inplace review
- Смена ролями по окончании блока кода или по времени

Ping-pong

- Самый сложный подход из всех парных
- Основан на TDD



Один пишет тест, второй реализует код для прохождения теста
Смена ролей



Вопрос!

Что первично: тест или код?

```
[ OK ] PULSETest.power_80_percent (0 ms)
[ RUN ] PULSETest.power_200_percent_truncated_to_100
[ OK ] PULSETest.power_200_percent_truncated_to_100 (0 ms)
[ RUN ] PULSETest.uint32_limits
[ OK ] PULSETest.uint32_limits (344 ms)
[ RUN ] PULSETest.change_power
[ OK ] PULSETest.change_power (0 ms)
[ RUN ] PULSETest.change_period
[ OK ] PULSETest.change_period (0 ms)
[ RUN ] PULSETest.zero_period
[ OK ] PULSETest.zero_period (0 ms)
[ RUN ] PULSETest.zero_power
[ OK ] PULSETest.zero_power (0 ms)
[ RUN ] PULSETest.change_power_in_a_middle
[ OK ] PULSETest.change_power_in_a_middle (0 ms)
[-----] 14 tests from PULSETest (356 ms total)

[-----] 3 tests from PULSEHigherTest
[ RUN ] PULSEHigherTest.powerless
[ OK ] PULSEHigherTest.powerless (0 ms)
[ RUN ] PULSEHigherTest.first_pulse
[ OK ] PULSEHigherTest.first_pulse (0 ms)
[ RUN ] PULSEHigherTest.power_80_percent
[ OK ] PULSEHigherTest.power_80_percent (0 ms)
[-----] 3 tests from PULSEHigherTest (2 ms total)

[-----] 1 test from SPIBUS_BasicTest
[ RUN ] SPIBUS_BasicTest.can_create_spibus
[ OK ] SPIBUS_BasicTest.can_create_spibus (0 ms)
[-----] 1 test from SPIBUS_BasicTest (0 ms total)
```

TDD

Test driven development

Код начинается с теста

[Non compilable] - Выражение идеи функции или алгоритма. Тест содержит описание алгоритма без реализации.

[Fail] Тестируемый код не выполняет ожидаемого действия.

[Pass] Написан **минимальный** функционал обеспечивающий выполнение теста.

```
TEST_F(GCodeFileConverterTest, open_ram_failure)
{
    std::string command = "G0 X0 Y0";
    createFile("file.gcode", command.c_str(), command.size());
    m_ram->SetCardStatus(SDCARD_NOT_READY);
    ASSERT_EQ(0, FileManagerOpenGCode(m_file_manager, "file.gcode"));
}

TEST_F(GCodeFileConverterTest, read_valid_gcode_out_of_scope)
{
    std::string command = "G0 X0 Y0";
    createFile("file.gcode", command.c_str(), command.size());
    FileManagerOpenGCode(m_file_manager, "file.gcode");
    FileManagerReadGCodeBlock(m_file_manager);
    ASSERT_EQ(PRINTER_FILE_NOT_GCODE, FileManagerReadGCodeBlock(m_file_manager));
}

TEST_F(GCodeFileConverterTest, read_valid_long_line)
{
    std::string command = "G0 X0 Y0 ";
    for (uint32_t i = 0; i < SDCARD_BLOCK_SIZE; ++i)
    {
        command += "A";
    }
    createFile("file.gcode", command.c_str(), command.size());
    FileManagerOpenGCode(m_file_manager, "file.gcode");
    FileManagerReadGCodeBlock(m_file_manager);
    ASSERT_EQ(PRINTER_FILE_NOT_GCODE, FileManagerReadGCodeBlock(m_file_manager));
}

TEST_F(GCodeFileConverterTest, read_sdcard_failure)
{
    std::string command = "G0 X0 Y0";
    createFile("file.gcode", command.c_str(), command.size());
    FileManagerOpenGCode(m_file_manager, "file.gcode");
    m_sdcard->SetCardStatus(SDCARD_NOT_READY);
    ASSERT_EQ(PRINTER_SDCARD_FAILURE, FileManagerReadGCodeBlock(m_file_manager));
}
```

TDD

Test driven development

Атомарный тест

- Один тест - одна проверка
- Максимально простой сценарий
- Независимые тесты, каждый тест начинается с “чистого стеята”
- Исключение внешних факторов
- В сложных случаях использование Setup/Teardown

Зачем

- Хороший набор тестов - лучшая документация
- Упрощение отладки и изолирование проблем
- Исключение наведенных ошибок
- Упрощение разработки кода
- Безопасный рефакторинг

С точки зрения ROI: Тактическое удорожание
Стратегическое удешевление

```
TEST_F(GCodeFileConverterTest, open_ram_failure)
{
    std::string command = "G0 X0 Y0";
    createFile("file.gcode", command.c_str(), command.size());
    m_ram->SetCardStatus(SDCARD_NOT_READY);
    ASSERT_EQ(0, FileManagerOpenGCode(m_file_manager, "file.gcode"));
}

TEST_F(GCodeFileConverterTest, read_valid_gcode_out_of_scope)
{
    std::string command = "G0 X0 Y0";
    createFile("file.gcode", command.c_str(), command.size());
    FileManagerOpenGCode(m_file_manager, "file.gcode");
    FileManagerReadGCodeBlock(m_file_manager);
    ASSERT_EQ(PRINTER_FILE_NOT_GCODE, FileManagerReadGCodeBlock(m_file_manager));
}

TEST_F(GCodeFileConverterTest, read_valid_long_line)
{
    std::string command = "G0 X0 Y0 ";
    for (uint32_t i = 0; i < SDCARD_BLOCK_SIZE; ++i)
    {
        command += "A";
    }
    createFile("file.gcode", command.c_str(), command.size());
    FileManagerOpenGCode(m_file_manager, "file.gcode");
    FileManagerReadGCodeBlock(m_file_manager);
    ASSERT_EQ(PRINTER_FILE_NOT_GCODE, FileManagerReadGCodeBlock(m_file_manager));
}

TEST_F(GCodeFileConverterTest, read_sdcard_failure)
{
    std::string command = "G0 X0 Y0";
    createFile("file.gcode", command.c_str(), command.size());
    FileManagerOpenGCode(m_file_manager, "file.gcode");
    m_sdcard->SetCardStatus(SDCARD_NOT_READY);
    ASSERT_EQ(PRINTER_SDCARD_FAILURE, FileManagerReadGCodeBlock(m_file_manager));
}
```

Вопрос!

Изолированный тест, как это?

Сила окружает тебя, почувствуй ее в воде, камнях

Mock тестирование (GMOCK)

- Контролируемое окружение
- Исключение внешнего влияния на тестируемый компонент
- Проверка сценариев взаимодействия с внешними компонентами input/output
- Имитация модулей, которые невозможно подключить к тесту (Drivers, HW, OS)



Пофехтуем?

```
TEST(GT_LS, simple_less)
```

```
{
```

```
    ASSERT_TRUE(compare
```

```
}
```

```
TEST(GT_LS, simple_greater)
```

```
{
```

```
    ASSERT_FALSE(compare
```

```
}
```

```
TEST(GT_LS, corner_cases)
```

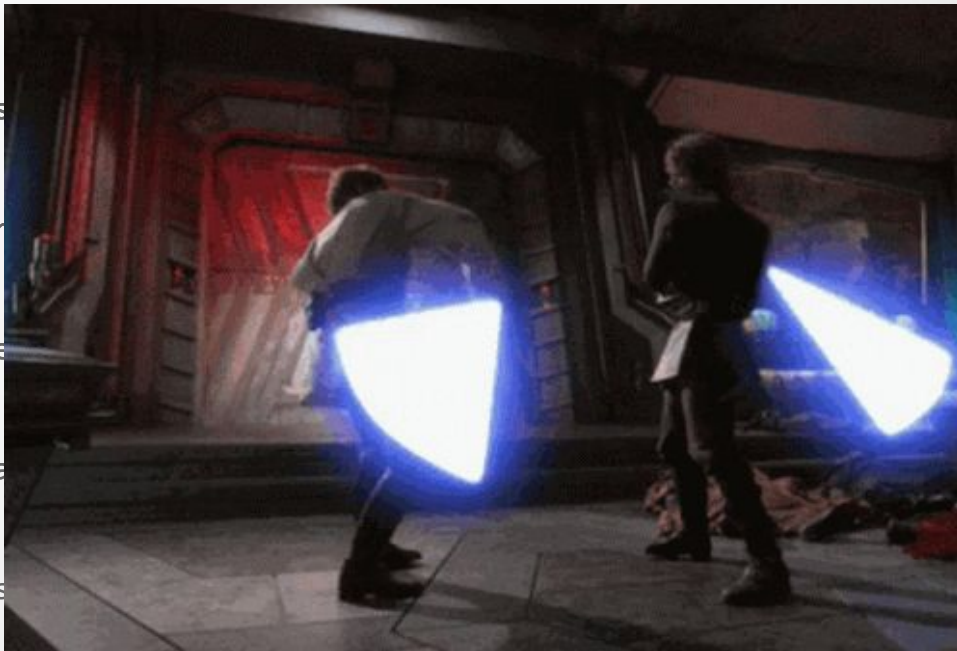
```
{
```

```
    ASSERT_FALSE(compare_less(2,2)) << "2 точно не меньше чем 2";
```

```
}
```

```
bool compare_less(int value, int threshold)
{
```

```
    return value < threshold;
```





← To Be Continued